Support Vector Machine & Its Applications

A portion (1/3) of
the slides are taken from
Prof. Andrew Moore's
SVM tutorial at
http://www.cs.cmu.edu/~awm/tutorials

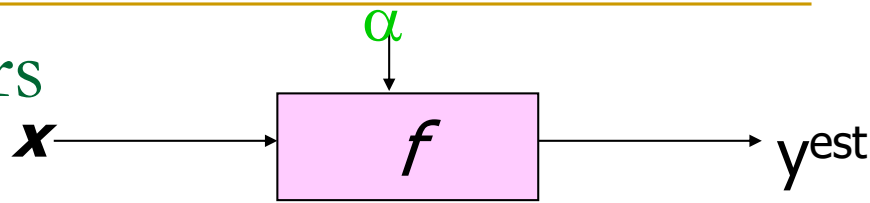
Mingyue Tan

The University of British Columbia

Outline

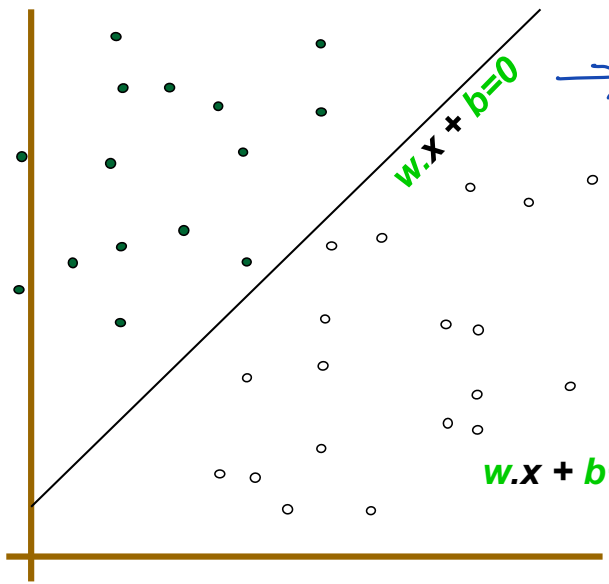
- Introduction to support vector machines
- Hard margin SVM
- Soft margin SVM
- Kernel trick for non-linear SVM
- Multiclass SVM

Linear Classifiers



- denotes +1
- denotes -1

$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}.\mathbf{x} + b)$



$\underline{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$

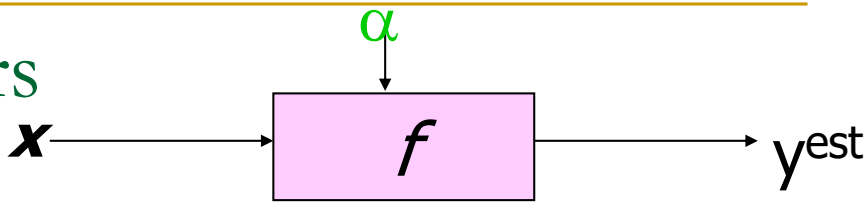
$\underline{w} \cdot \underline{x} + b = 0$

How would you classify this data?

$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b = 0$

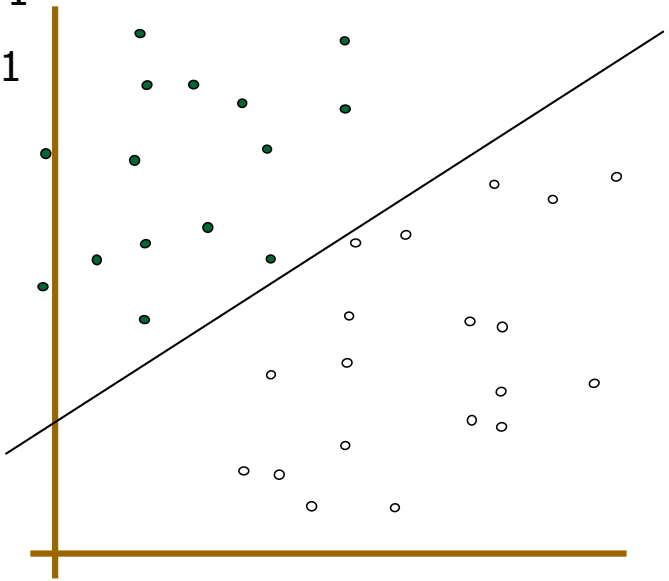
$w_1 x_1 + w_2 x_2 + b = 0$

Linear Classifiers



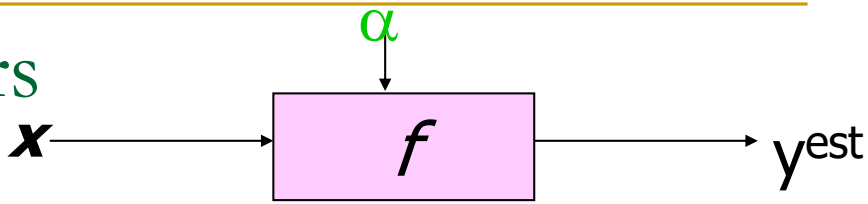
$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}.\mathbf{x} + b)$

- denotes +1
- denotes -1



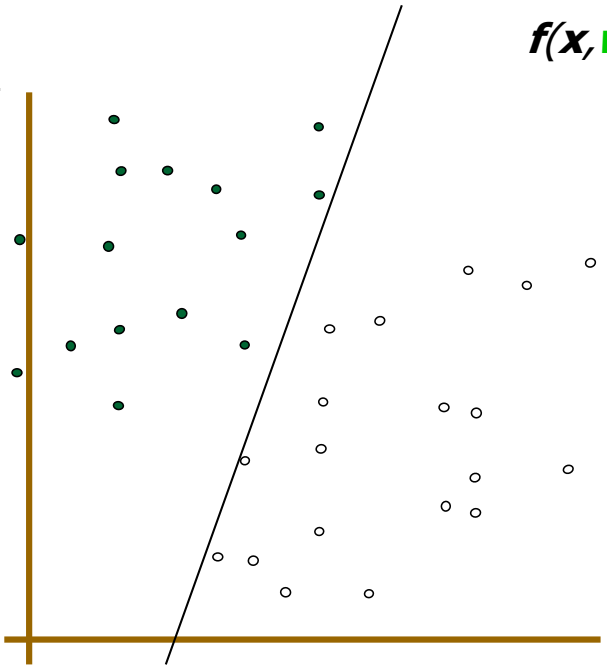
How would you classify this data?

Linear Classifiers



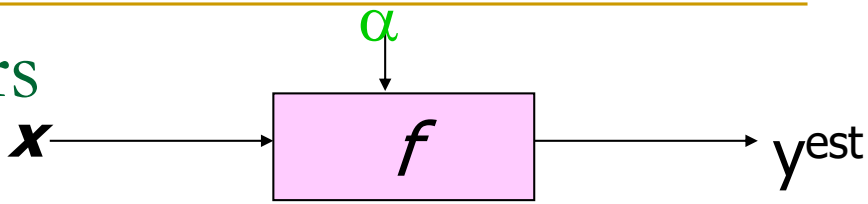
$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}.\mathbf{x} + b)$

- denotes +1
- denotes -1



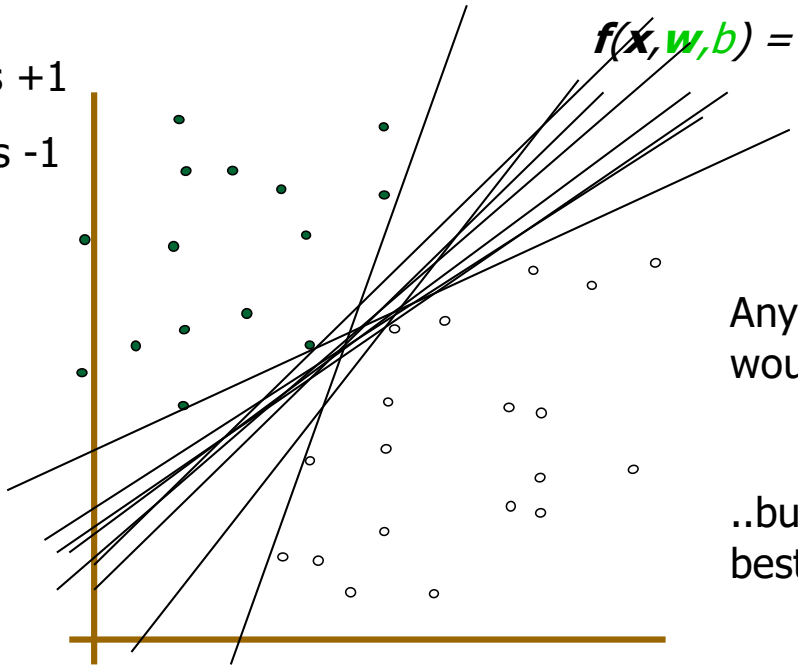
How would you classify this data?

Linear Classifiers



$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}.\mathbf{x} + b)$

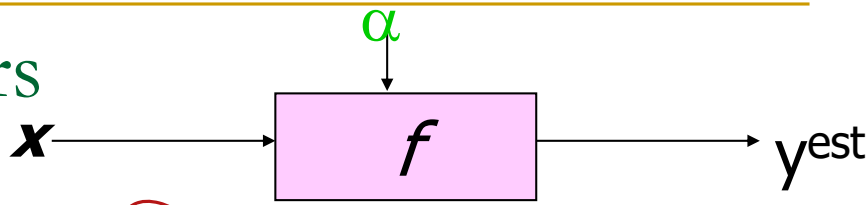
- denotes +1
- denotes -1



Any of these would be fine..

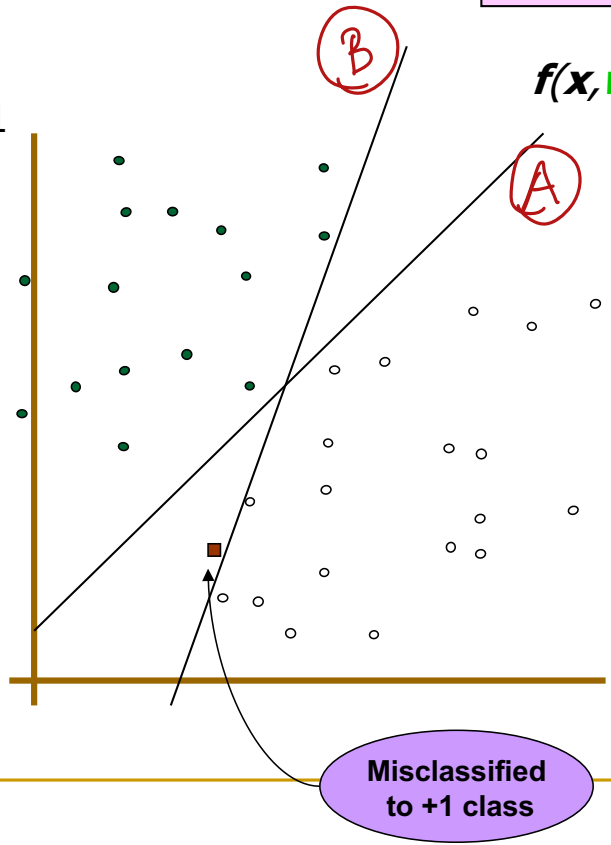
..but which is best?

Linear Classifiers



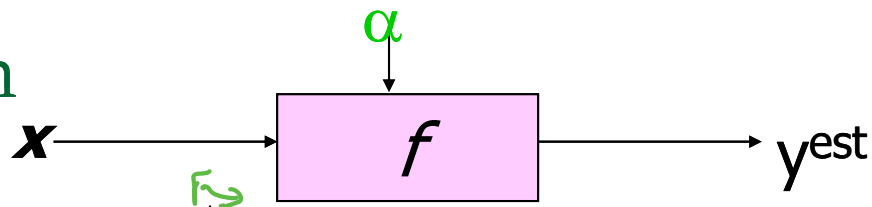
$f(\mathbf{x}, \mathbf{w}, b) = sign(\mathbf{w}.\mathbf{x} + b)$

- denotes +1
- denotes -1

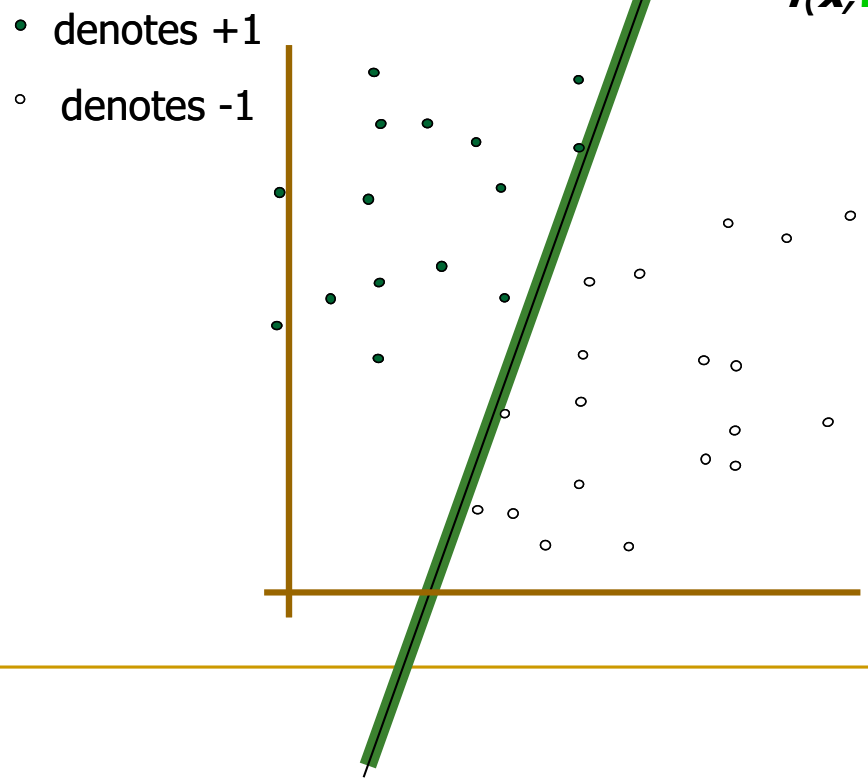


How would you classify this data?

Classifier Margin

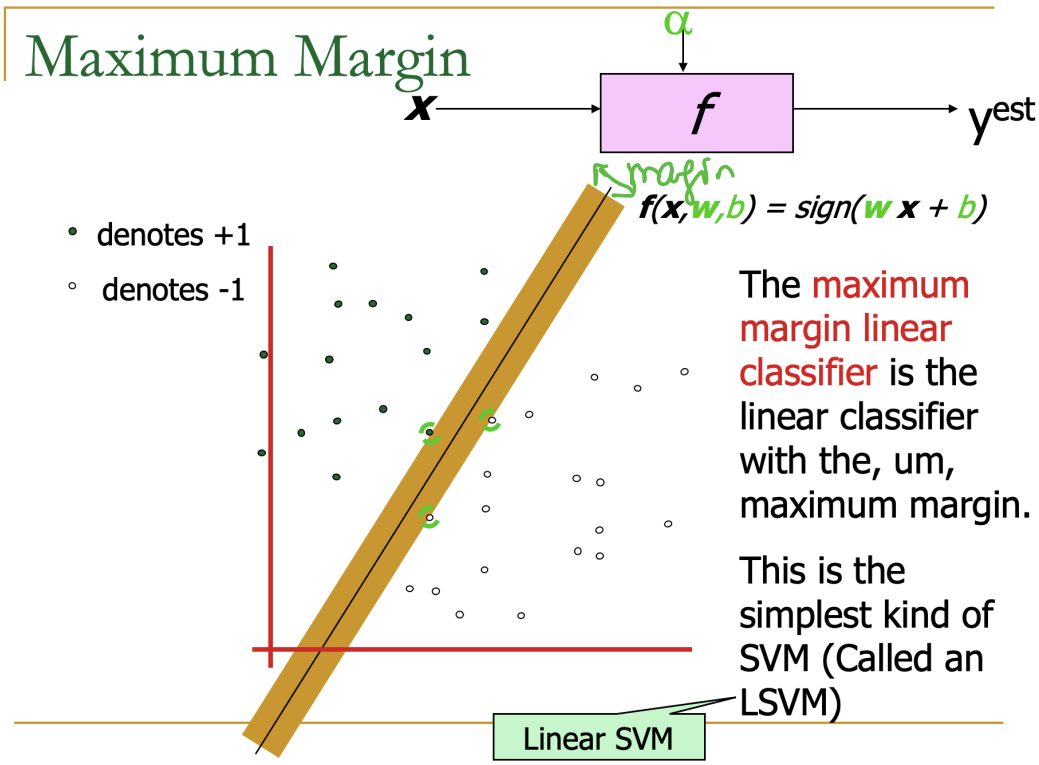


$f(\mathbf{x},\mathbf{w},b) = sign(\mathbf{w}.\mathbf{x} + b)$

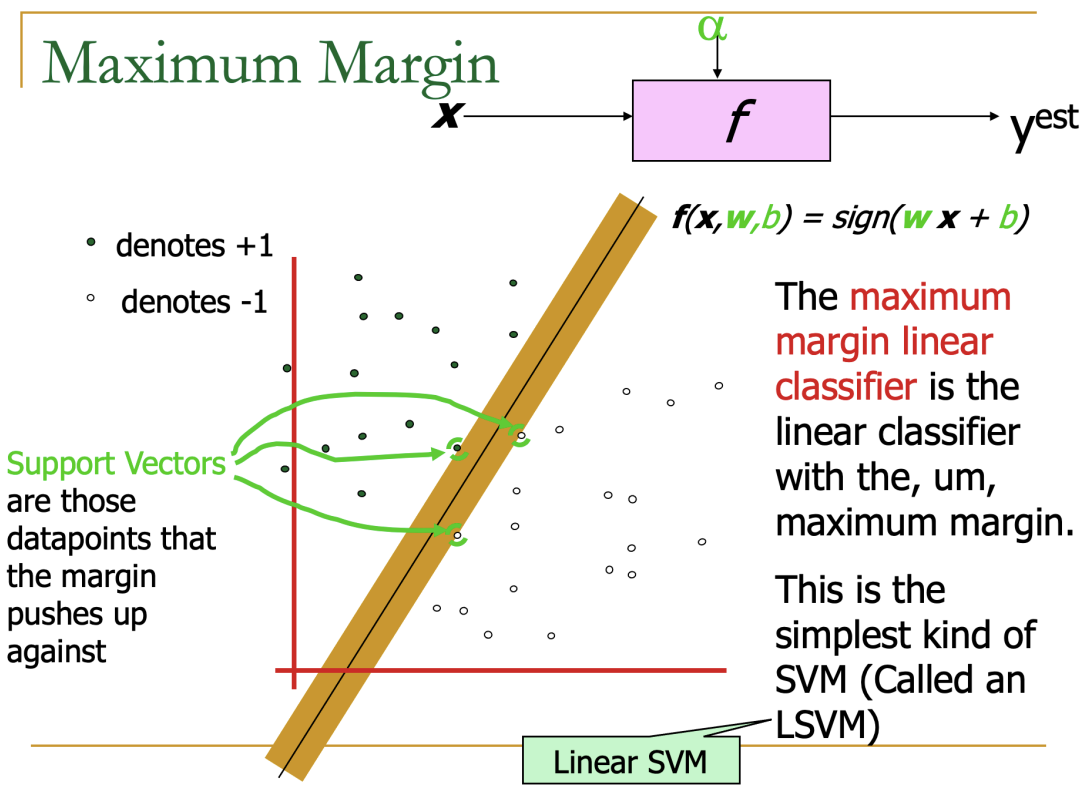


Define the margin of a linear classifier as the width that the boundary could be increased by before hitting a datapoint.

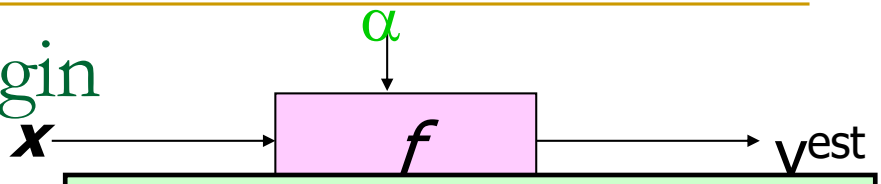
Maximum Margin



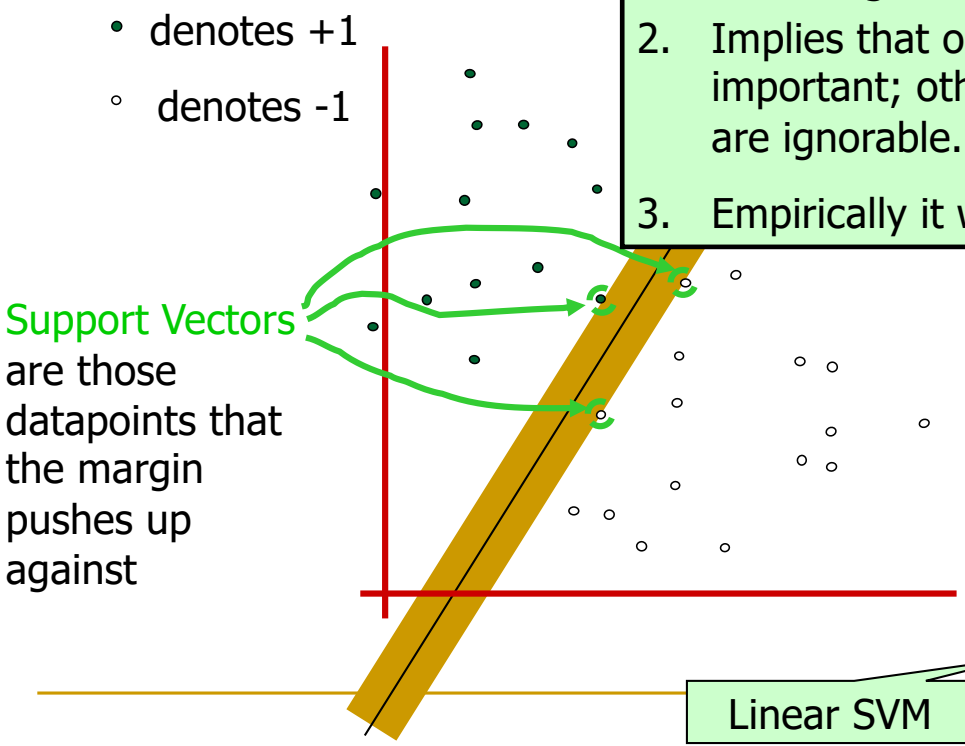
Maximum Margin



Maximum Margin



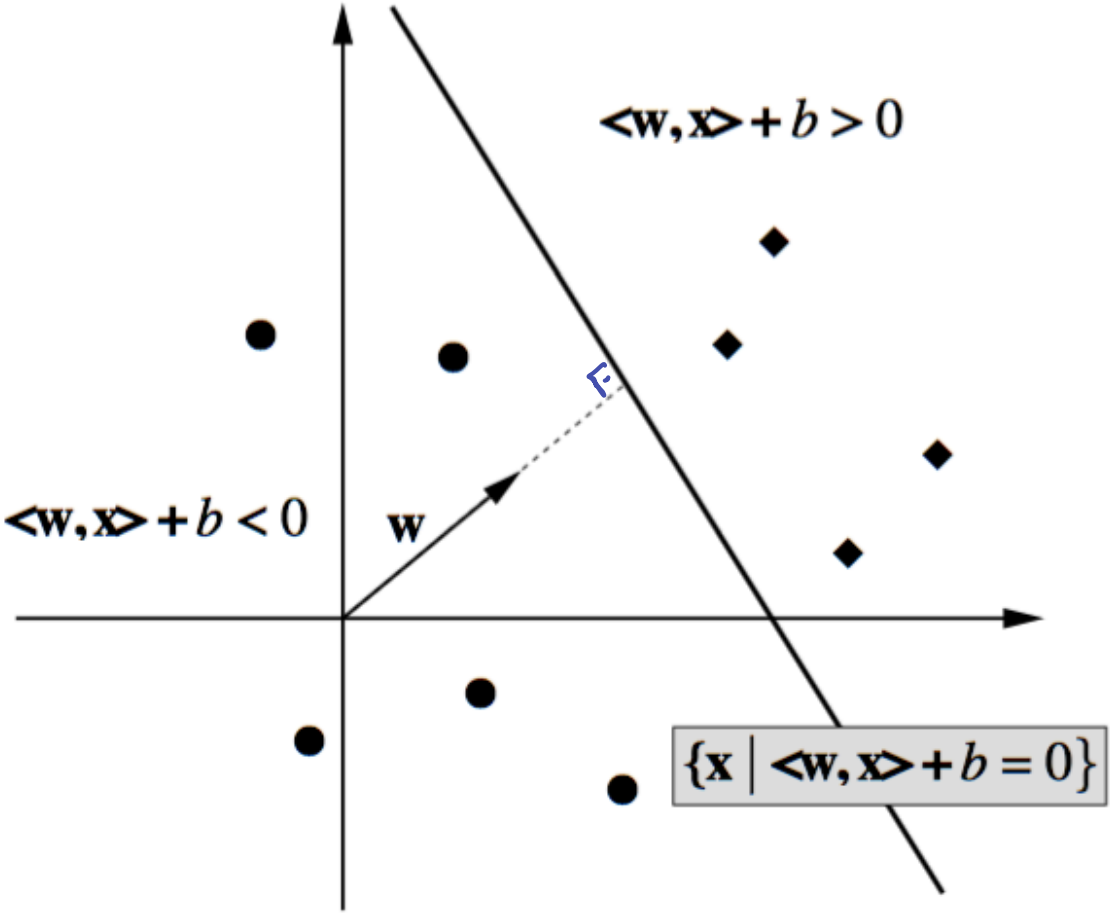
1. Maximizing the margin is good according to intuition and PAC theory
2. Implies that only support vectors are important; other training examples are ignorable.
3. Empirically it works very very well.

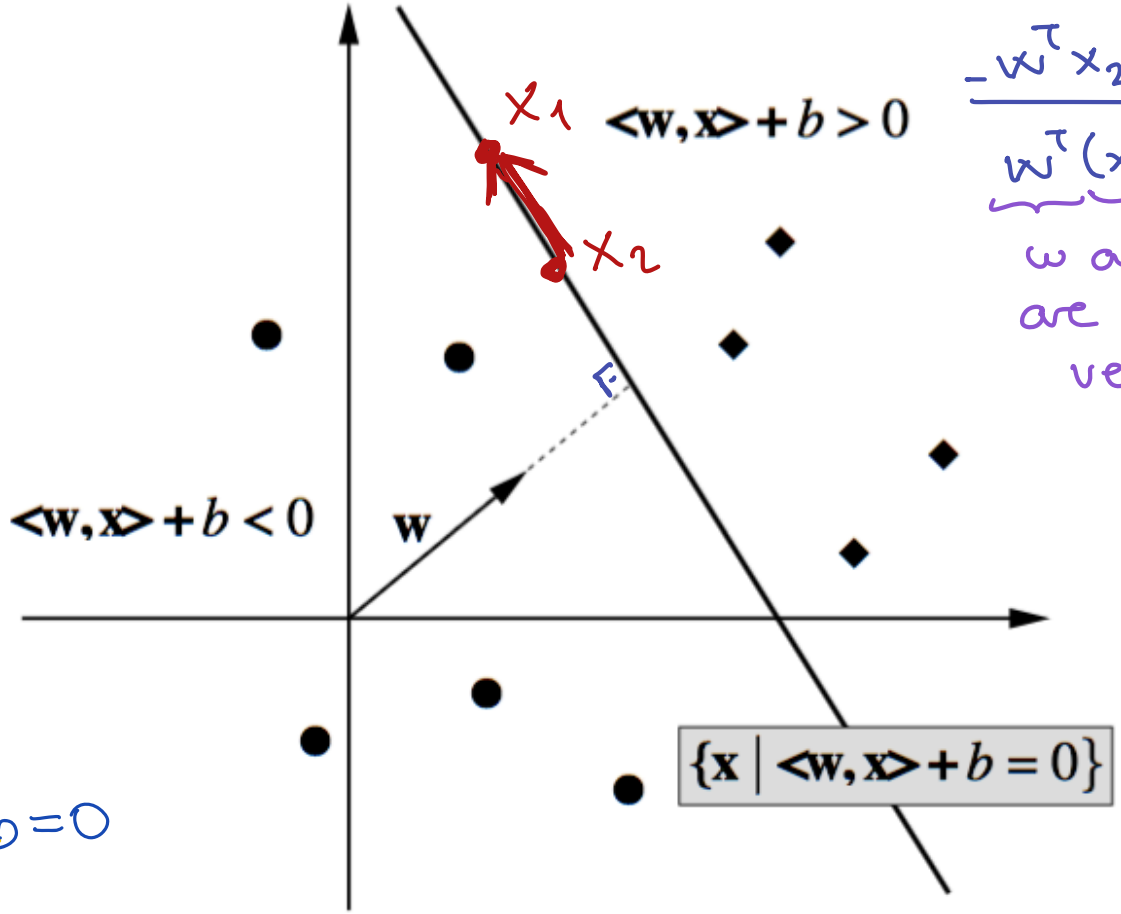


linear classifier with the, um, maximum margin.

This is the simplest kind of SVM (Called an LSVM)

Linear SVM


$\langle \mathbf{w}, \mathbf{x} \rangle + b > 0$
$\langle \mathbf{w}, \mathbf{x} \rangle + b < 0$
$\mathbf{w}$
$\{\mathbf{x} \mid \langle \mathbf{w}, \mathbf{x} \rangle + b = 0\}$



$w^T x_1 + b = 0$

$-w^T x_2 + b = 0$

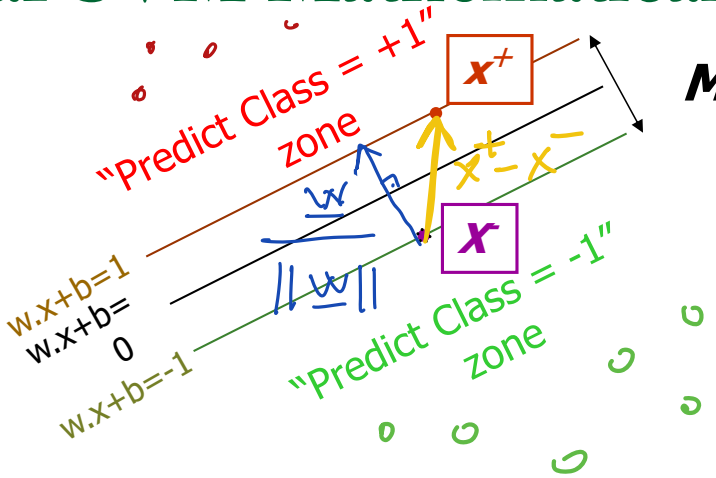
$w^T (x_1 - x_2) = 0$

w and $(x_1 - x_2)$ are orthogonal vectors.

x_1 and x_2 are on the line.

$a w^T x + a b = 0$

Linear SVM Mathematically



M=Margin Width

What we know:

- $\boldsymbol{w} \cdot \boldsymbol{x}^+ + b = +1$
- $\boldsymbol{w} \cdot \boldsymbol{x}^- + b = -1$
- $\boldsymbol{w} \cdot (\boldsymbol{x}^+ - \boldsymbol{x}^-) = 2$

$$M = \frac{(x^+ - x^-) \cdot w}{|w|} = \frac{2}{|w|}$$

Linear SVM Mathematically

- Goal: **1) Correctly classify all training data**

$$wx_i + b \geq 1 \quad \text{if } y_i = +1$$
$$wx_i + b \leq -1 \quad \text{if } y_i = -1$$
$$y_i(wx_i + b) \geq 1 \quad \text{for all i}$$

2) Maximize the Margin

$$M = \frac{2}{|w|}$$

same as minimize

$$\frac{1}{2} w^t w$$

- **We can formulate a Quadratic Optimization Problem and solve for w and b**

- Minimize $\Phi(w) = \frac{1}{2} w^t w$

subject to $y_i(wx_i + b) \geq 1 \qquad \forall i$

Solving the Optimization Problem

Find **w** and b such that
$\Phi(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w}$ is minimized;
and for all $\{(\mathbf{x_i}, y_i)\}$: $y_i\,(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

- **Need to optimize a *quadratic* function subject to *linear* constraints.**
- **Quadratic optimization problems are a well-known class of mathematical programming problems, and many (rather intricate) algorithms exist for solving them.**
- **The solution involves constructing a *dual problem* where a *Lagrange multiplier* α_i is associated with every constraint in the primary problem:**

Find $\alpha_1 \ldots \alpha_N$ such that
$\mathbf{Q}(\boldsymbol{\alpha}) = \sum\alpha_i - \frac{1}{2}\sum\sum\alpha_i\alpha_j y_i y_j \mathbf{x_i}^T\mathbf{x_j}$ is maximized and
(1) $\sum\alpha_i y_i = 0$
(2) $\alpha_i \geq 0$ for all α_i

The Optimization Problem Solution

- The solution has the form:

$$\mathbf{w} = \Sigma \alpha_i y_i \mathbf{x_i} \qquad b = y_k - \mathbf{w}^T \mathbf{x_k} \text{ for any } \mathbf{x_k} \text{ such that } \alpha_k \neq 0$$

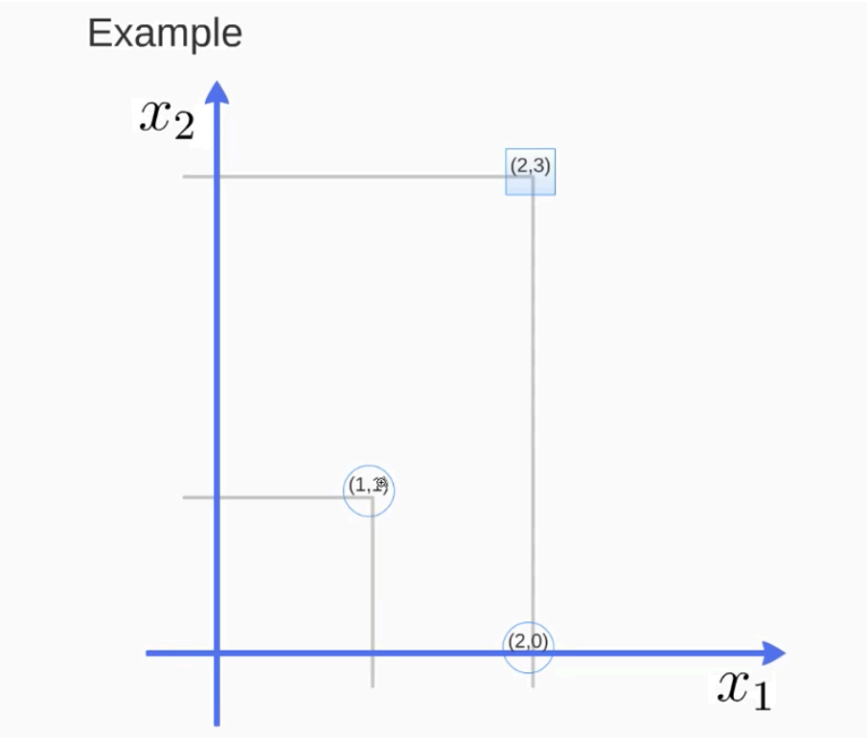
- Each non-zero α_i indicates that corresponding $\mathbf{x_i}$ is a support vector.
- Then the classifying function will have the form:

$$f(\mathbf{x}) = \Sigma \alpha_i y_i \mathbf{x_i}^T \mathbf{x} + b$$

$w^T x + b$

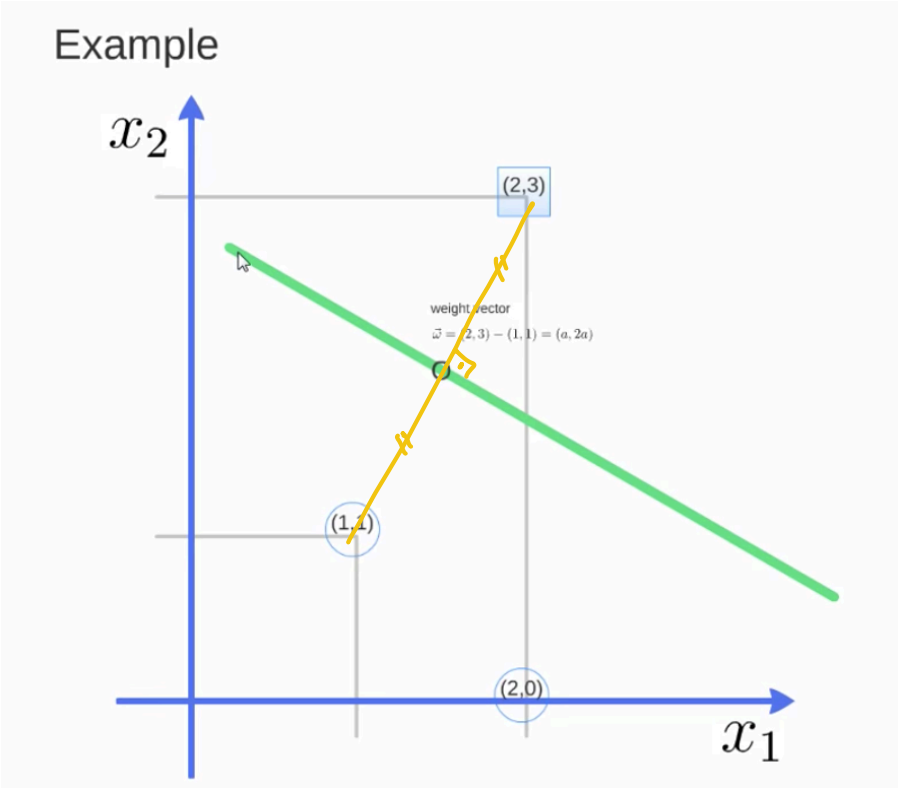
- Notice that it relies on an *inner product* between the test point $\mathbf{x}$ and the support vectors $\mathbf{x_i}$ – we will return to this later.
- Also keep in mind that solving the optimization problem involved computing the inner products $\mathbf{x_i}^T \mathbf{x_j}$ between all pairs of training points.

Example

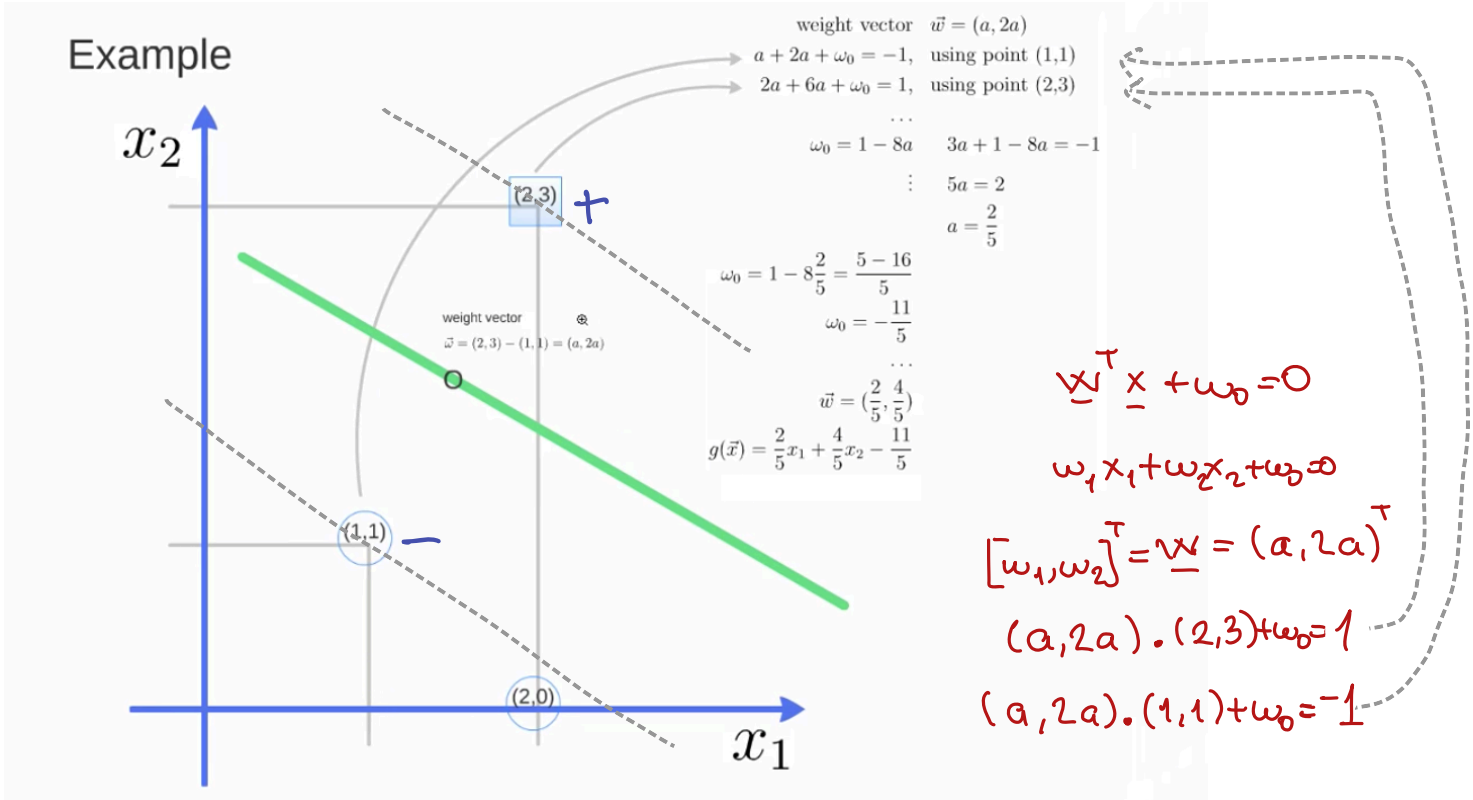


Design a linear SVM classifier.

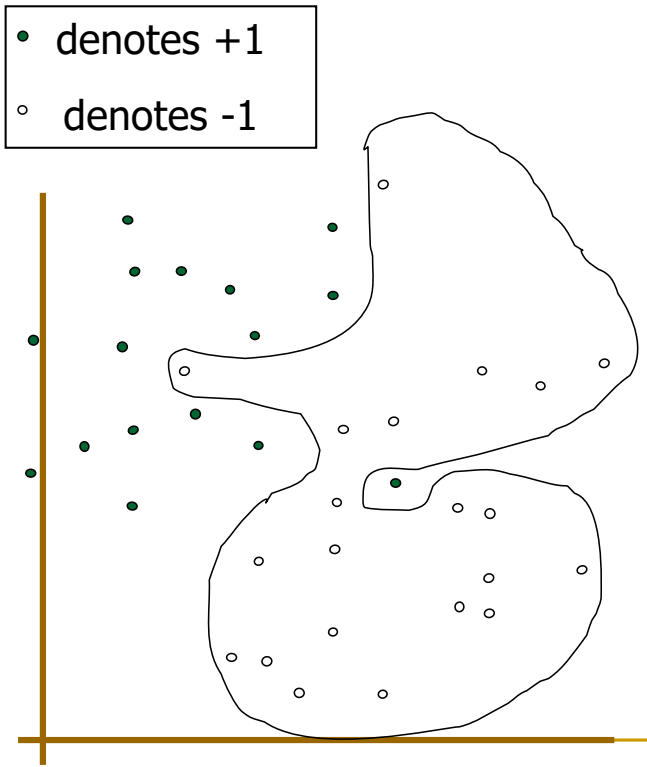
Example



Example


Example
x_2
x_1
(2,3)
(1,1)
(2,0)
weight vector
$\vec{w} = (2,3) - (1,1) = (a, 2a)$
weight vector $\vec{w} = (a, 2a)$
$a + 2a + \omega_0 = -1$, using point (1,1)
$2a + 6a + \omega_0 = 1$, using point (2,3)
$\omega_0 = 1 - 8a$ $3a + 1 - 8a = -1$
$5a = 2$
$a = \frac{2}{5}$
$\omega_0 = 1 - 8\frac{2}{5} = \frac{5-16}{5}$
$\omega_0 = -\frac{11}{5}$
$\vec{w} = (\frac{2}{5}, \frac{4}{5})$
$g(\vec{x}) = \frac{2}{5}x_1 + \frac{4}{5}x_2 - \frac{11}{5}$
$\underline{w}^T \underline{x} + \omega_0 = 0$
$\omega_1 x_1 + \omega_2 x_2 + \omega_0 = 0$
$[\omega_1, \omega_2]^T = \underline{w} = (a, 2a)^T$
$(a, 2a) \cdot (2,3) + \omega_0 = 1$
$(a, 2a) \cdot (1,1) + \omega_0 = -1$

Dataset with noise



- **Hard Margin: So far we require all data points be classified correctly**

  - No training error

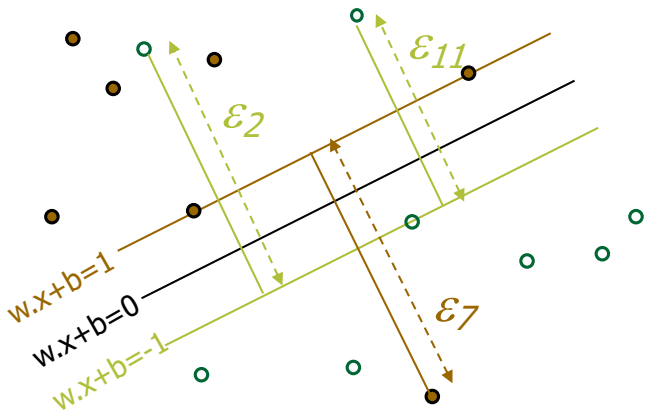
- **What if the training set is noisy?**

  - Solution 1: use very powerful kernels

OVERFITTING!

Soft Margin Classification

***Slack variables ξi* can be added to allow misclassification of difficult or noisy examples.**



What should our quadratic optimization criterion be?

Minimize

$$\frac{1}{2}\mathbf{w}.\mathbf{w} + C\sum_{k=1}^{R}\varepsilon_k$$

$\underline{w}.\underline{x}_i + b \geq 1-\xi_i$, if $y_i = +1$

$\underline{w}.\underline{x}_i + b \leq -1+\xi_i$, if $y_i = -1$

} Combine $y_i(\underline{w}.\underline{x}_i + b) \geq 1-\xi_i$

Hard Margin v.s. Soft Margin

- **The old formulation:**

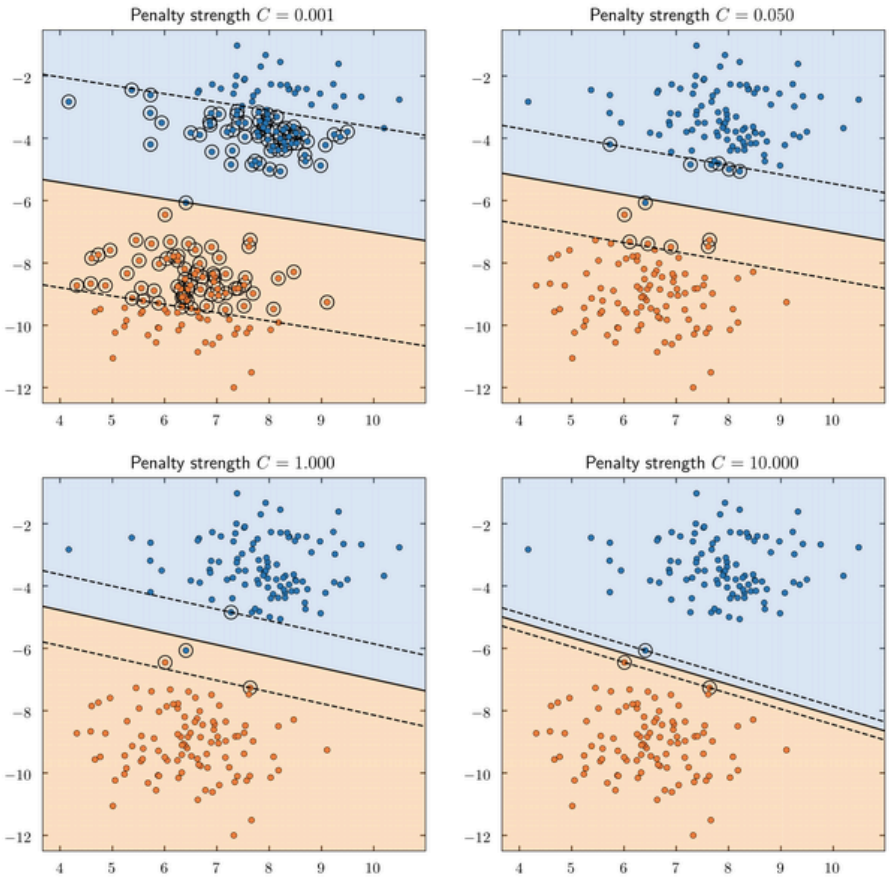
Find $\mathbf{w}$ and b such that
$\Phi(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w}$ is minimized and for all $\{(\mathbf{x_i}, y_i)\}$
$y_i\,(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$

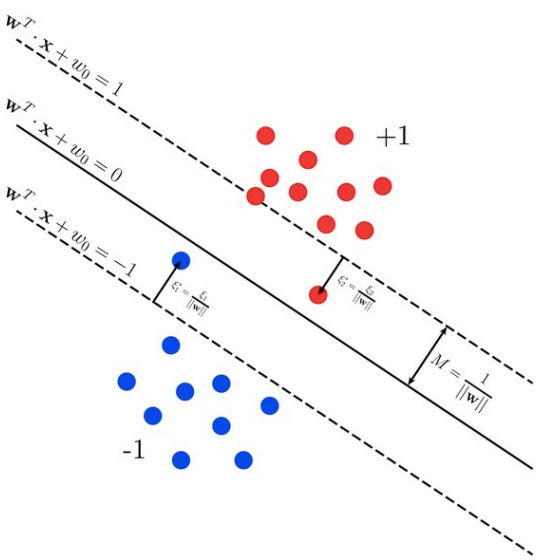
- **The new formulation incorporating slack variables:**

Find $\mathbf{w}$ and b such that
$\Phi(\mathbf{w}) = \frac{1}{2}\,\mathbf{w}^T\mathbf{w} + C\sum\xi_i$ is minimized and for all $\{(\mathbf{x_i}, y_i)\}$
$y_i\,(\mathbf{w}^T\mathbf{x_i} + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i

- **Parameter *C* can be viewed as a way to control overfitting.**

The effect of the penalty strength is shown below.





Linear SVMs: Overview

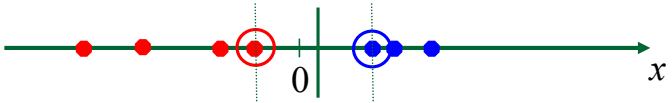
- The classifier is a *separating hyperplane.*
- Most "important" training points are support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points x_i are support vectors with non-zero Lagrangian multipliers α_i.
- Both in the dual formulation of the problem and in the solution training points appear only inside dot products:

Find $\alpha_1 \ldots \alpha_N$ such that
$Q(\alpha) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and
(1) $\Sigma\alpha_i y_i = 0$
(2) $0 \le \alpha_i \le C$ for all α_i

$$f(\mathbf{x}) = \Sigma\alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

Non-linear SVMs

- Datasets that are linearly separable with some noise work out great:

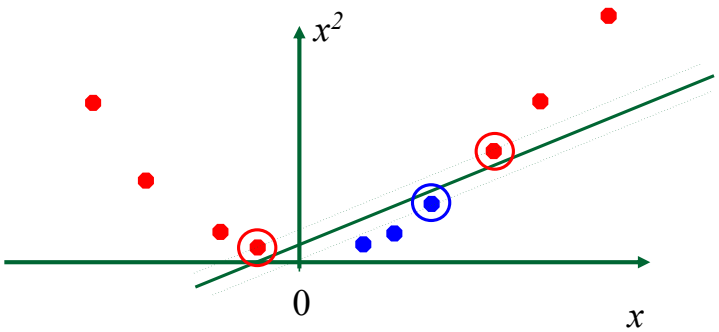


- But what are we going to do if the dataset is just too hard?



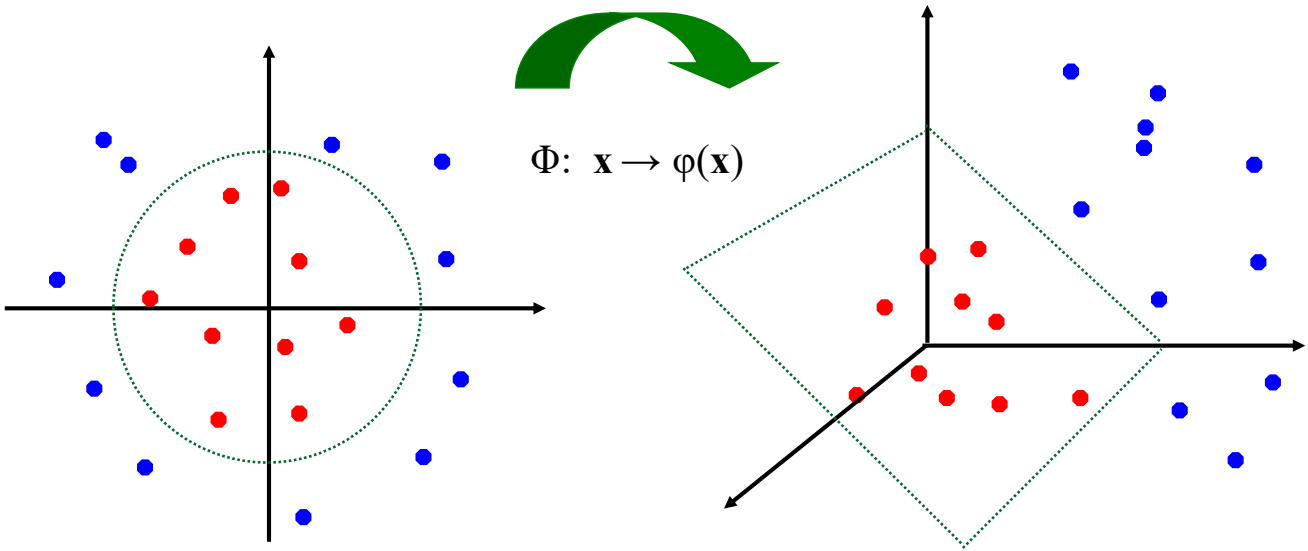
- How about… mapping data to a higher-dimensional space:

$x \xrightarrow{\Phi(x)} [x, x^2]$



Non-linear SVMs: Feature spaces

- General idea: the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



The "Kernel Trick"

- **The linear classifier relies on dot product between vectors $K(x_i,x_j)=x_i^T x_j$**
- **If every data point is mapped into high-dimensional space via some transformation Φ: $x \rightarrow \varphi(x)$, the dot product becomes:**

$$K(x_i,x_j)= \varphi(x_i)^T \varphi(x_j)$$

- **A *kernel function* is some function that corresponds to an inner product in some expanded feature space.**
- **Example:**

2-dimensional vectors $x=[x_1 \ x_2]$; let $K(x_i,x_j)=(1 + x_i^T x_j)^2$,

Need to show that $K(x_i,x_j)= \varphi(x_i)^T \varphi(x_j)$:

$$K(x_i,x_j)=(1 + x_i^T x_j)^2, = \left(1+ \begin{bmatrix} x_{i1} & x_{i2} \end{bmatrix} \begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix}\right)^2 = (1+ x_{i1}x_{j1} + x_{i2}x_{j2})^2$$

$$= 1+ x_{i1}^2x_{j1}^2 + 2\, x_{i1}x_{j1}\, x_{i2}x_{j2}+ x_{i2}^2x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2}$$

$$= \begin{bmatrix} 1 & x_{i1}^2 & \sqrt{2}\, x_{i1}x_{i2} & x_{i2}^2 & \sqrt{2}x_{i1} & \sqrt{2}x_{i2} \end{bmatrix}^T \begin{bmatrix} 1 & x_{j1}^2 & \sqrt{2}\, x_{j1}x_{j2} & x_{j2}^2 & \sqrt{2}x_{j1} & \sqrt{2}x_{j2} \end{bmatrix}$$

$$= \varphi(x_i)^T \varphi(x_j), \quad \text{where } \varphi(x) = [1 \ x_1^2 \ \sqrt{2}\, x_1x_2 \ x_2^2 \ \sqrt{2}x_1 \ \sqrt{2}x_2]$$

27 Polynomial-SVMs

The kernel $K(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}')^d$ gives the same result as the explicit mapping + dot product that we described before:

$$\Phi : R^2 \to R^3 \quad (x_1, x_2) \mapsto (z_1, z_2, z_3) := (x_1^2, \sqrt{(2)} x_1 x_2, x_2^2)$$

$$\Phi((x_1, x_2) \cdot \Phi((x_1', x_2') = (x_1^2, \sqrt{(2)} x_1 x_2, x_2^2) \cdot (x'^2_1, \sqrt{(2)} x'_1 x'_2, x'^2_2)$$

$$\underline{x} \qquad \underline{x'} = x_1^2 x_1'^2 + 2 x_1 x'_1 x_2 x'_2 + x_2^2 x_2'^2$$

is the same as:

$$K(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x} \cdot \boldsymbol{x}')^2 = ((x_1, x_2) \cdot (x'_1, x'_2))^2$$

$$= (x_1 x'_1 + x_2 x'_2)^2 = x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2 x_1 x'_1 x_2 x'_2$$

Interestingly, if d is large the kernel is still only requires n multiplications to compute, whereas the explicit representation may not fit in memory!

Can be inner product in infinite dimensional space

Assume $x \in R^1$ and $\gamma > 0$.

$$e^{-\gamma\|x_i - x_j\|^2} = e^{-\gamma(x_i - x_j)^2} = e^{-\gamma x_i^2 + 2\gamma x_i x_j - \gamma x_j^2} = e^{-\gamma x_i^2 - \gamma x_j^2} \cdot e^{2\gamma x_i x_j}$$

$$= e^{-\gamma x_i^2 - \gamma x_j^2}\left(1 + \frac{2\gamma x_i x_j}{1!} + \frac{(2\gamma x_i x_j)^2}{2!} + \frac{(2\gamma x_i x_j)^3}{3!} + \cdots\right)$$

$$= e^{-\gamma x_i^2 - \gamma x_j^2}\left(1 \cdot 1 + \sqrt{\frac{2\gamma}{1!}} x_i \cdot \sqrt{\frac{2\gamma}{1!}} x_j + \sqrt{\frac{(2\gamma)^2}{2!}} x_i^2 \cdot \sqrt{\frac{(2\gamma)^2}{2!}} x_j^2 \right.$$

$$\left. + \sqrt{\frac{(2\gamma)^3}{3!}} x_i^3 \cdot \sqrt{\frac{(2\gamma)^3}{3!}} x_j^3 + \cdots\right) = \phi(x_i)^T \phi(x_j),$$

where

$$\phi(x) = e^{-\gamma x^2}\left[1, \sqrt{\frac{2\gamma}{1!}} x, \sqrt{\frac{(2\gamma)^2}{2!}} x^2, \sqrt{\frac{(2\gamma)^3}{3!}} x^3, \cdots\right]^T.$$



- So what kind of kernel should I use?
- What kind of functions are valid kernels?
- How to decide kernel parameters?

What Functions are Kernels?

- **For some functions $K(x_i,x_j)$ checking that**

  $K(x_i,x_j)= \varphi(x_i)^T \varphi(x_j)$ can be cumbersome.

- **Mercer's theorem:**

  Every semi-positive definite symmetric function is a kernel

- **Semi-positive definite symmetric functions correspond to a semi-positive definite symmetric Gram matrix:**

K=

$K(x_1,x_1)$	$K(x_1,x_2)$	$K(x_1,x_3)$	…	$K(x_1,x_N)$
$K(x_2,x_1)$	$K(x_2,x_2)$	$K(x_2,x_3)$		$K(x_2,x_N)$
…	…	…	…	…
$K(x_N,x_1)$	$K(x_N,x_2)$	$K(x_N,x_3)$	…	$K(x_N,x_N)$

Examples of Kernel Functions

- Linear: $K(\mathbf{x_i},\mathbf{x_j}) = \mathbf{x_i}^T\mathbf{x_j}$
- Polynomial of power p: $K(\mathbf{x_i},\mathbf{x_j}) = (1+ \mathbf{x_i}^T\mathbf{x_j})^p$
- Gaussian (radial-basis function network):

$$K(\mathbf{x_i},\mathbf{x_j}) = \exp\left(-\frac{\left\|\mathbf{x_i}-\mathbf{x_j}\right\|^2}{2\sigma^2}\right)$$

- Sigmoid: $K(\mathbf{x_i},\mathbf{x_j}) = \tanh(\beta_0\mathbf{x_i}^T\mathbf{x_j} + \beta_1)$

Non-linear SVMs Mathematically

- **Dual problem formulation:**

Find $\alpha_1 \ldots \alpha_N$ such that
$Q(\alpha) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j K(x_i, x_j)$ is maximized and
(1) $\Sigma\alpha_i y_i = 0$
(2) $\alpha_i \geq 0$ for all α_i

- **The solution is:**

$$f(x) = \Sigma\alpha_i y_i K(x_i, x_j) + b$$

- **Optimization techniques for finding α_i's remain the same!**

Decision function

- At optimum

$$\mathbf{w} = \sum_{i=1}^{l} \alpha_i y_i \phi(\mathbf{x}_i)$$

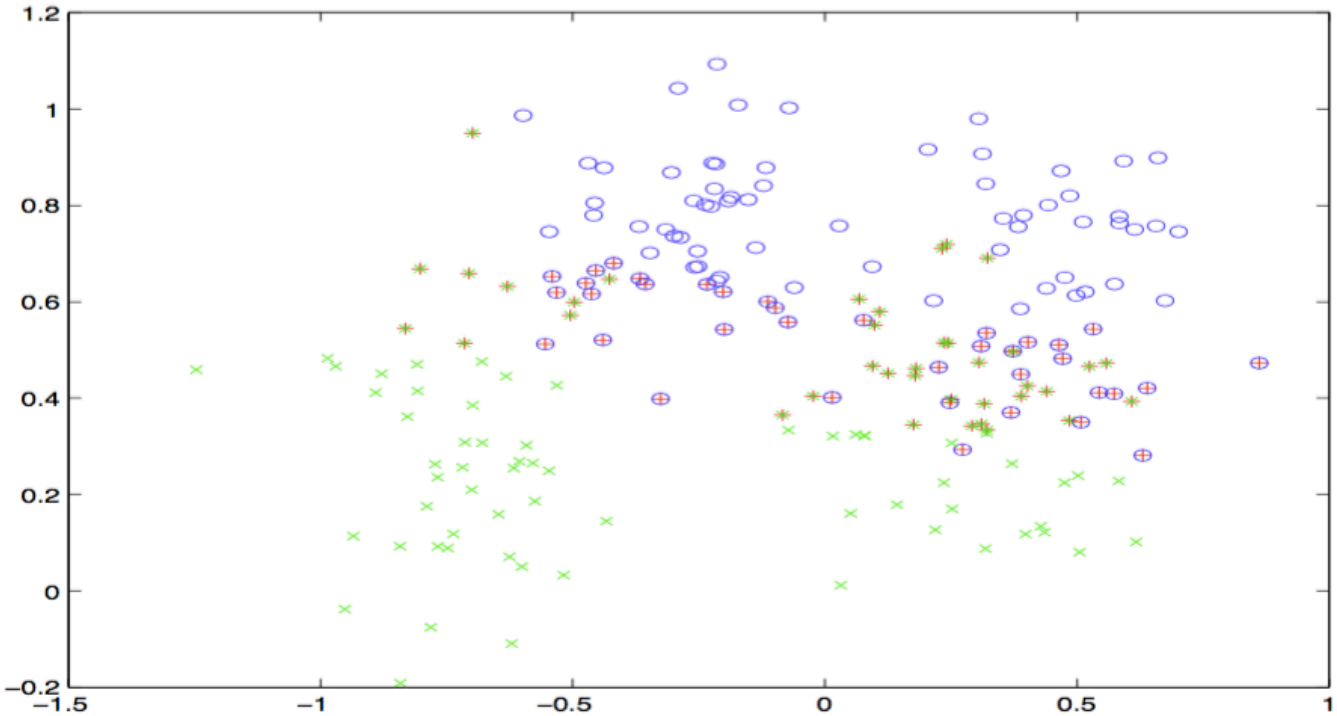
- Decision function

$$\begin{aligned} & \mathbf{w}^T \phi(\mathbf{x}) + b \\ = & \sum_{i=1}^{l} \alpha_i y_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}) + b \\ = & \sum_{i=1}^{l} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b \end{aligned}$$

- Only $\phi(\mathbf{x}_i)$ of $\alpha_i > 0$ used $\Rightarrow$ support vectors



Support Vectors: More Important Data



Nonlinear SVM - Overview

- SVM locates a separating hyperplane in the feature space and classify points in that space
- It does not need to represent the space explicitly, simply by defining a kernel function
- The kernel function plays the role of the dot product in the feature space.

Properties of SVM

- **Flexibility in choosing a similarity function**
- **Sparseness of solution when dealing with large data sets**

  - only support vectors are used to specify the separating hyperplane
- **Ability to handle large feature spaces**

  - complexity does not depend on the dimensionality of the feature space
- **Overfitting can be controlled by soft margin approach**
- **Nice math property: a simple convex optimization problem which is guaranteed to converge to a single global solution**

Weakness of SVM

- **It is sensitive to noise**

  - A relatively small number of mislabeled examples can dramatically decrease the performance

- **It only considers two classes**

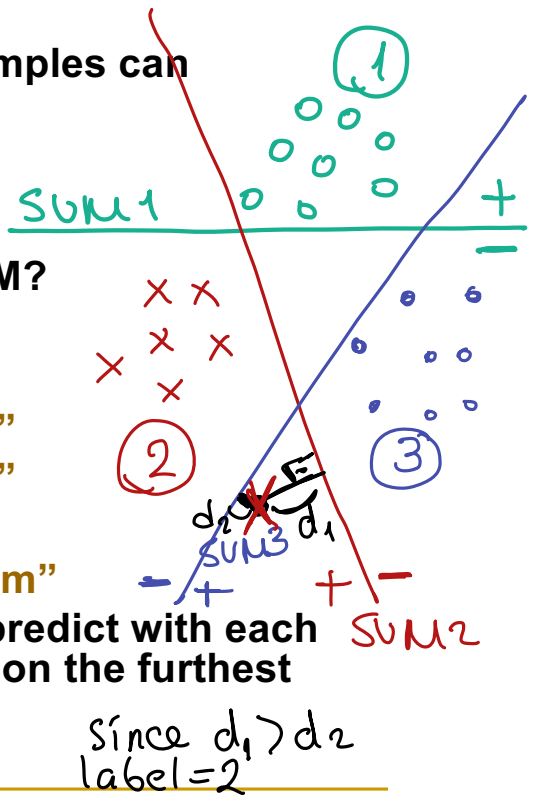
  - how to do multi-class classification with SVM?

  - Answer:

  1) with output parity m, learn m SVM's

  - **SVM 1 learns "Output==1" vs "Output != 1"**
  - **SVM 2 learns "Output==2" vs "Output != 2"**
  - **:**
  - **SVM m learns "Output==m" vs "Output != m"**

  2)To predict the output for a new input, just predict with each SVM and find out which one puts the prediction the furthest into the positive region.



SVM Applications

- **SVM has been used successfully in many real-world problems**
  - **text (and hypertext) categorization**
  - **image classification**
  - **bioinformatics (Protein classification, Cancer classification)**
  - **hand-written character recognition**

22 SVMs : non-linear case II

For example MNIST hand-writing recognition.
60,000 training examples, 10000 test examples, 28x28.

Linear SVM has around 8.5% test error.
Polynomial SVM has around 1% test error.



Some Issues

- **Choice of kernel**
  - **Gaussian or polynomial kernel is default**
  - **if ineffective, more elaborate kernels are needed**
  - **domain experts can give assistance in formulating appropriate similarity measures**

- **Choice of kernel parameters**
  - **e.g. σ in Gaussian kernel**
  - **σ is the distance between closest points with different classifications**
  - **In the absence of reliable criteria, applications rely on the use of a validation set or cross-validation to set such parameters.**

- **Optimization criterion** – Hard margin v.s. Soft margin
  - **a lengthy series of experiments in which various parameters are tested**

30 SVMs : software

Lots of SVM software:

- LibSVM (C++)
- SVMLight (C)

As well as complete machine learning toolboxes that include SVMs:

- Torch (C++)
- Spider (Matlab)
- Weka (Java)

All available through `www.kernel-machines.org`.

A Simple Procedure

1. Conduct simple scaling on the data
2. Consider RBF kernel $K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x}-\mathbf{y}\|^2}$
3. Use cross-validation to find the best parameter C and γ
4. Use the best C and γ to train the whole training set
5. Test

For beginners only, you can do a lot more

Multi-class Classification

- k classes
- One-against-the rest: Train k binary SVMs:

$$\begin{array}{lll} \text{1st class} & \text{vs.} & (2-k)\text{th class} \\ \text{2nd class} & \text{vs.} & (1,3-k)\text{th class} \\ & \vdots & \end{array}$$

- k decision functions

$$\begin{array}{c} (\mathbf{w}^1)^T\phi(\mathbf{x}) + b_1 \\ \vdots \\ (\mathbf{w}^k)^T\phi(\mathbf{x}) + b_k \end{array}$$

- Prediction:

$$\arg\max_j \ (\mathbf{w}^j)^T \phi(\mathbf{x}) + b_j$$

- Reason: If the 1st class, then we should have

$$\begin{aligned}
(\mathbf{w}^1)^T \phi(\mathbf{x}) + b_1 &\geq +1 \\
(\mathbf{w}^2)^T \phi(\mathbf{x}) + b_2 &\leq -1 \\
&\vdots \\
(\mathbf{w}^k)^T \phi(\mathbf{x}) + b_k &\leq -1
\end{aligned}$$

Multi-class Classification (Cont'd)

- One-against-one: train $k(k-1)/2$ binary SVMs $(1,2),(1,3),\ldots,(1,k),(2,3),(2,4),\ldots,(k-1,k)$
- If 4 classes $\Rightarrow$ 6 binary SVMs

$y_i = 1$	$y_i = -1$	Decision functions
class 1	class 2	$f^{12}(\mathbf{x}) = (\mathbf{w}^{12})^T\mathbf{x} + b^{12}$
class 1	class 3	$f^{13}(\mathbf{x}) = (\mathbf{w}^{13})^T\mathbf{x} + b^{13}$
class 1	class 4	$f^{14}(\mathbf{x}) = (\mathbf{w}^{14})^T\mathbf{x} + b^{14}$
class 2	class 3	$f^{23}(\mathbf{x}) = (\mathbf{w}^{23})^T\mathbf{x} + b^{23}$
class 2	class 4	$f^{24}(\mathbf{x}) = (\mathbf{w}^{24})^T\mathbf{x} + b^{24}$
class 3	class 4	$f^{34}(\mathbf{x}) = (\mathbf{w}^{34})^T\mathbf{x} + b^{34}$

- For a testing data, predicting all binary SVMs

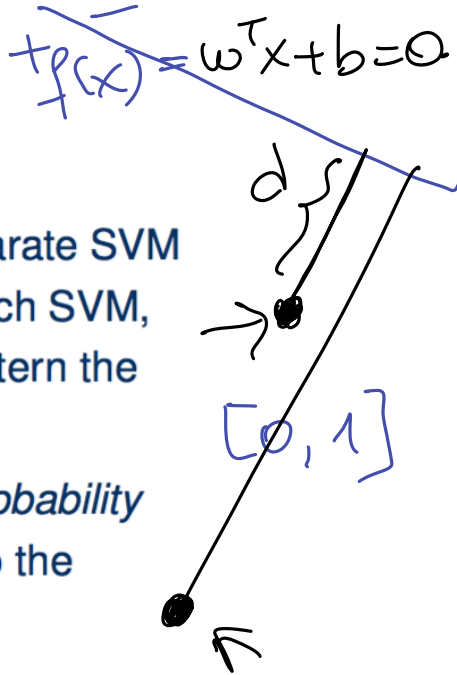
Classes		winner
1	2	1
1	3	1
1	4	1
2	3	2
2	4	4
3	4	3

- Select the one with the largest vote

class	1	2	3	4
# votes	3	1	1	1

- May use decision values as well

Support Vector Machines



- *Extension to multiple classes:* We can train a separate SVM for each class, compute the output value using each SVM, and select the class that assigns the unknown pattern the furthest into the positive region.
- *Converting the output of an SVM to a posterior probability for post-processing:* We can fit a sigmoid model to the posterior probability $P(y = 1|f(\mathbf{x}))$ as

$$P(y = 1|f(\mathbf{x})) = \frac{1}{1 + \exp(a\, f(\mathbf{x}) + b)}$$

where the parameters a and b are learned using maximum likelihood estimation from a training set.

Additional Resources http://www.kernel-machines.org/

- **An excellent tutorial on VC-dimension and Support Vector Machines:**

  C.J.C. Burges. A tutorial on support vector machines for pattern recognition. Data Mining and Knowledge Discovery, 2(2):955-974, 1998.

- **The VC/SRM/SVM Bible:**

  Statistical Learning Theory by Vladimir Vapnik, Wiley-Interscience; 1998

  - A 3-D demonstration
    www.csie.ntu.edu.tw/~cjlin/libsvmtools/svmtoy3d
  - Further references, for example,
    [Cristianini and Shawe-Taylor, 2000,
    Schölkopf and Smola, 2002]

Reference

- **Support Vector Machine Classification of Microarray Gene Expression Data**, Michael P. S. Brown William Noble Grundy, David Lin, Nello Cristianini, Charles Sugnet, Manuel Ares, Jr., David Haussler
- www.cs.utexas.edu/users/mooney/cs391L/svm.**ppt**
- **Text categorization with Support Vector Machines:**
  learning with many relevant features

  T. Joachims, ECML - 98

  A great resource is the MLSS 2006 (Machine Learning Summer School) talk by Chih-Jen Lin, available at: videolectures.net/mlss06tw lin svm/